

Software Requirements Specification (SRS)

Project Title: Cargo Office Management System

Table of Contents

Software Requirements Specification (SRS).....	1
Project Title: Cargo Office Management System.....	2
1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Project Milestones & Schedule.....	4
2. Overall Description.....	4
2.1 Product Perspective	4
2.2 Product Functions	4
2.3 User Classes and Characteristics	5
3. Functional Requirements.....	5
3.1 Authentication & User Management	5
3.2 Operations Department	5
3.3 Finance Department.....	5
3.4 Customer Service Department.....	6
3.5 Warehouse / Logistics Department	6
4. Non-Functional Requirements	6
4.1 Performance Requirements	6
4.2 Security Requirements	6
4.3 Usability	6
4.4 Reliability and Availability.....	6
4.5 Maintainability	7
4.6 Portability	7
5. System Features.....	7
6. Use Case Modeling	7

6.1 Use Case: User Login	7
6.2 Use Case: Update Cargo Status.....	7
6.3 Use Case: Generate Invoice	7
7. Data Requirements	8
7.1 Data Dictionary	8
8. Technologies Used.....	8
8.1 Front-End Technologies	8
8.2 Back-End Technologies	8
8.3 Database	8
8.4 Authentication & Security	8
8.5 Deployment	9
8.6 Testing Tools	9
8.7 Optional Tools & Libraries	9

1. Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements for the Cargo Office Management System. This system is designed to automate, streamline, and manage the core operations of a cargo company, including logistics, finance, customer service, and warehouse management.

1.2 Scope

The Cargo Office Management System will support the following key functionalities:

- Cargo booking, scheduling, and shipment tracking
- Invoicing, manual payment recording, and financial reporting
- Customer profile management, inquiry handling, and complaint resolution
- Warehouse space allocation, incoming/outgoing cargo tracking, and inventory management
- Role-based access and comprehensive data reporting

1.3 Project Milestones & Schedule

The development lifecycle is guided by the following deadlines:

- **SRS Documentation:** Deadline 22 - 24
- **Start UI Design:** Deadline 24 - 26 or 27
- **Start Frontend Development:** Deadline 29 - July 3
- **Start Backend Development:** Deadline 6 - 17

2. Overall Description

2.1 Product Perspective

This system is a standalone web-based application. It will interface with a backend database and provide a user-friendly GUI for different departments, administrators, and customer service teams.

Suggested System Pages include: Login Page, Dashboard, Cargo Management Page, Shipment Tracking Page, Finance Management Page, Customer Management Page, Warehouse Management Page, Reports Page, Notifications Page, Settings Page, and Contact/Support Page.

2.2 Product Functions

- Manage cargo bookings and transport routes
- Track real-time delivery statuses
- Generate invoices, receipts, and financial summaries
- Manage customer profiles and handle inquiries
- Track warehouse inventory and cargo damages

2.3 User Classes and Characteristics

- **Admin:** Full access to the system, including all modules, user management, and advanced reports.
- **Operations Staff:** Manage cargo bookings, scheduling, tracking, and transport allocation.
- **Finance Staff:** Handle invoices, manual payments, expenses, and financial reports.
- **Customer Service Staff:** Manage customer registrations, inquiries, complaints, and feedback.
- **Warehouse/Logistics Staff:** Manage incoming/outgoing cargo, space allocation, and stock tracking.

3. Functional Requirements

3.1 Authentication & User Management

- User login and logout functionality.
- Role-based access control (RBAC) to restrict features based on departments.
- Admin user management (create, edit, delete users).
- Password reset functionality.
- Secure authentication system protocols.

3.2 Operations Department

- Cargo booking management.
- Shipment scheduling system.
- Cargo tracking management.
- Delivery status updates.
- Route and transport management.
- Vehicle/container allocation.
- Shipment history records maintenance.
- Generate operational reports.

3.3 Finance Department

- Invoice generation.
- Manual payment recording.
- Expense management.
- Billing management.
- Financial report generation.
- Transaction history management.
- Tax calculation support.
- Receipt generation.
- Payment status management (Paid / Pending).
- Cash and bank transfer record management.
- *(Note: Online payment gateway integration is not included in this phase).*

3.4 Customer Service Department

- Customer registration and profile management.
- Inquiry handling system.
- Complaint management.
- Shipment status checking.
- Customer feedback management.
- Notification support (Email/SMS alerts).
- Customer communication records.

3.5 Warehouse / Logistics Department

- Inventory and cargo storage management.
- Incoming cargo recording.
- Outgoing cargo management.
- Warehouse space allocation.
- Cargo damage/lost reporting.
- Stock tracking system.
- Barcode/QR code support.
- Logistics report generation.

4. Non-Functional Requirements

4.1 Performance Requirements

- Fast system performance with optimized load times.
- Support for multiple concurrent users across different departments.
- Response time < 2 seconds for most standard operations.

4.2 Security Requirements

- Secure data handling and secure login sessions.
- Role-based access control to protect cross-departmental data.
- Encrypted passwords and sensitive data.
- Activity logs and audit tracking.

4.3 Usability

- Intuitive, user-friendly interface requiring minimal training.
- Mobile responsive design to accommodate users on various devices.

4.4 Reliability and Availability

- High system availability (99.9% uptime).
- Reliable database management ensuring data consistency.
- Data backup and recovery support.

4.5 Maintainability

- Scalable architecture to support future business growth.
- Modular codebase with clear documentation.

4.6 Portability

- Deployable on standard Linux or Windows servers.
- Cloud-ready architecture.

5. System Features

- Multi-role login authentication.
- Advanced search, filter, and sorting functionality across all modules.
- Export data functionalities (Excel, PDF for reports).
- Centralized Dashboard with analytics, statistics, and KPIs.
- Notifications and alerts (in-system and email/SMS).
- Comprehensive Activity logs.

6. Use Case Modeling

6.1 Use Case: User Login

- **Actor:** Admin / Staff User
- **Precondition:** Valid login credentials.
- **Description:** User logs into the system to access their specific departmental dashboard and modules based on their assigned role.

6.2 Use Case: Update Cargo Status

- **Actor:** Operations Staff
- **Precondition:** Cargo has been booked and dispatched.
- **Description:** Operations staff searches for the specific shipment ID and updates the current tracking status (e.g., "In Transit", "Delivered").

6.3 Use Case: Generate Invoice

- **Actor:** Finance Staff
- **Precondition:** Cargo booking is confirmed and pricing is calculated.
- **Description:** Finance staff views the pending payment, generates an invoice with tax calculations, and marks the status as 'Pending' or 'Paid' upon manual cash/bank transfer.

7. Data Requirements

7.1 Data Dictionary

- **User Table:** UserID, Name, Email, PasswordHash, Role, Department, Status.
- **Customer Table:** CustomerID, Name, ContactNo, Email, Address, RegistrationDate.
- **Cargo/Shipment Table:** ShipmentID, CustomerID, Origin, Destination, Weight, Status, DispatchDate, DeliveryDate.
- **Invoice Table:** InvoiceID, ShipmentID, TotalAmount, Tax, PaymentStatus, PaymentMethod, Date.
- **Warehouse/Inventory Table:** ItemID, ShipmentID, WarehouseLocation, StorageDate, Status (Safe/Damaged).

8. Technologies Used

8.1 Front-End Technologies

- React.js / Next.js - For building a dynamic, component-based single-page application (SPA).
- Bootstrap / Tailwind CSS - For responsive and modern UI design.
- React Router - For client-side routing/navigation.

8.2 Back-End Technologies

- Node.js - JavaScript runtime for server-side scripting.
- Express.js - Web framework for building RESTful APIs.
- Prisma - ORM for database interaction (if applicable).
- REST API - For communication between front-end and back-end.
- Multer - For handling file uploads (e.g., receipts, cargo documents).

8.3 Database

- MongoDB - NoSQL document-based database
- OR MySQL/SQLite for relational data storage (Shipments, Finances, Users).

8.4 Authentication & Security

- JWT (JSON Web Tokens) - For user authentication and secure session handling.
- BCrypt.js - For hashing passwords before storing them.
- Helmet.js - For securing HTTP headers.
- CORS - For handling cross-origin requests.
- HTTPS/SSL - For secure data transmission.

8.5 Deployment

- CI/CD - GitHub Actions or GitLab CI for continuous integration.
- Version Control: Git, GitHub / GitLab.

8.6 Testing Tools

- Postman - For testing API endpoints.
- Jest - For unit testing.
- Supertest - For testing Node.js/Express routes.

8.7 Optional Tools & Libraries

- Chart.js / Recharts - For dashboard data visualization (e.g., revenue trends, shipment statistics).
- Cloudinary / AWS S3 - For storing uploaded documents.
- Email API / Nodemailer - For sending automated alerts and notifications to customers.